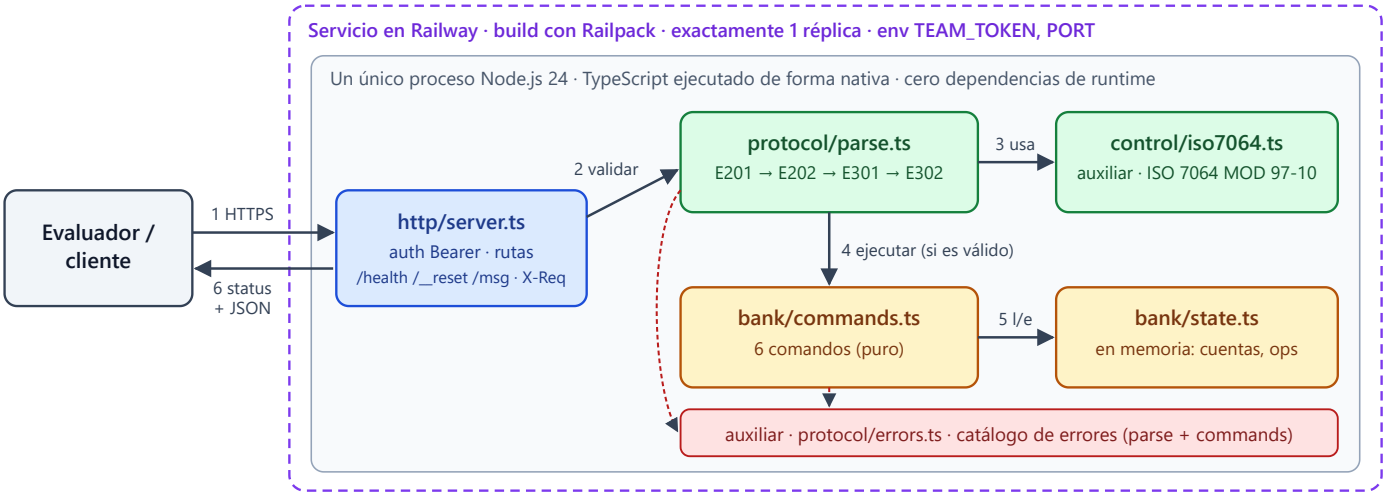
# Diseño de sistema — réplica del Reto 2 "La caja negra"

Reemplazo directo de un servicio sin documentación: mismo status HTTP y mismo cuerpo para cada solicitud, errores incluidos. Node.js 24 + TypeScript, `node:http`, cero dependencias de runtime, un único estado en memoria, desplegado como una sola instancia en Railway.




Flujo: 1 solicitud → 2 validar el mensaje → 3 verificar el valor de control → 4 ejecutar el comando → 5 leer/actualizar el estado → 6 respuesta. Etiquetas: observado respaldado por respuestas grabadas (`pruebas/ejemplos.jsonl`, `experimentos.jsonl`); supuesto valor por defecto no probado, aislado en el código como `// ASSUMPTION:`.

## Componentes y responsabilidades

- `http/server.ts` — único adaptador de E/S: autenticación Bearer, rutas, cuerpo JSON, contador `X-Req`.
- `protocol/parse.ts` — divide `m` por `;` (último campo = control); valida formato, valor de control, comando (sin distinguir mayúsculas) y aridad.
- `control/iso7064.ts` — puro: payload en mayúsculas, dígitos sin cambios, A=1…Z=26, separadores descartados; `98 - (N·100 mod 97)`.
- `bank/commands.ts` — PING, ALTA, CONSULTA, DEPOSITO, TRANSFER, ANULA; valida todas las entradas antes de modificar el estado.
- `bank/state.ts` — cuentas, operaciones, contadores `AC-NNNN` / `OP-NNNN`, reinicio.
- `protocol/errors.ts` — catálogo único de cuerpos exactos byte a byte (incl. `"recieved"` y el espacio final en `"comando desconocido "`).

## Estado y persistencia

- Solo en memoria, un único estado asociado a `TEAM_TOKEN`; `POST /__reset` lo limpia → `200 {"ok":true}` observado.
- Identificadores secuenciales desde 0001; las solicitudes rechazadas y ANULA no consumen identificador observado.
- ANULA de un TRANSFER debita el destino observado; acredita el origen supuesto.
- No se necesita persistencia: cada test de evaluación empieza con `POST /__reset` (README).

## Autenticación, validación y errores

- Token incorrecto → `401 E001 "no autorizado"` observado; sin token, lo mismo observado. `GET /health` exige token → `200 {"ok":true}` observado.
- Control: no numérico → `400 E201`; valor incorrecto → `400 E202` observado, fórmula ISO confirmada en los restos borde.
- Comando desconocido → `400 E301`; aridad incorrecta → `400 E302`; cuenta inexistente → `404 E401`; límite → `422 E506`; fondos → `422 E507` observado.
- Ruta inexistente → `404 E404` antes de la autenticación; body no JSON → `400 E100`; sin `m` → `400 E102`; monto inválido → `400 E303`; límite `> 100000` por transferencia observado.
- Orden E201 → E202 → E301 → E302 y código `E500` supuesto.

## Decisiones técnicas

- Eliminación nativa de tipos de TypeScript: sin paso de build, arranque rápido.
- `control` / `bank` puros + un único adaptador HTTP: lógica testeable sin servidor.
- Las pruebas golden reproducen los 20 ejemplos grabados y el experimento ISO (`npm test`: 11/11 aprobadas); `tsc --noEmit` sin errores.
- Sin LLM dentro de la réplica: el comportamiento es determinista. Nunca llama al original.
- Contrato de ingeniería en `AGENTS.md` (tipos estrictos, sin secretos, Definition of Done).

## Despliegue — una sola instancia

- Railway, builder Railpack (sin Docker): `node src/main.ts`, sin health check de plataforma (`/health` exige token), reinicio `ALWAYS`, Node `24.x`.
- Exactamente 1 réplica, sin autoescalado ni suspensión: el estado vive en el proceso y cada paso tiene 5 s.
- `TEAM_TOKEN` como variable del servicio, nunca versionado ni registrado en logs.

## Riesgos y limitaciones

- El orden de errores no probado y los supuestos restantes pueden fallar en tests con varios errores o casos borde.
- El original puede cambiar durante el día; la paridad se verifica contra la versión de las 20:00.
- Un reinicio pierde el estado a mitad de una secuencia; se mitiga con una única instancia siempre activa.